Systemd timers в Linux

Современное планирование задач простыми словами

cron → at → systemd timers

LPIC-1 • systemctl • journalctl • OnCalendar



Как работает systemd timer

Timer отвечает за расписание, service — за действие

backup.timer

Когда запускать

OnCalendar=
OnBootSec=
OnUnitActiveSec=

backup.service

Что выполнять

ExecStart=
/usr/local/bin/backup.sh

Команда / скрипт

Реальная работа:
backup, очистка, отчёт,
синхронизация...

По умолчанию `backup.timer` активирует `backup.service` с тем же именем.

Зачем timers, если уже есть cron?

Не замена ради замены — другой уровень интеграции

systemctl

Запуск, остановка, enable, status — всё теми же командами systemd.

journalctl

Логи задачи доступны как логи обычного systemd service.

Зависимости

Можно использовать Before/After, Requires и другие возможности unit-файлов.

От загрузки

OnBootSec и OnUnitActiveSec позволяют считать время относительно работы системы.

Persistent=true

Календарный timer может догнать запуск, пропущенный пока система была выключена.

Диагностика

list-timers + systemd-analyze calendar помогают быстро проверить расписание.

Cron остаётся отличным инструментом для простых задач.

Смотрим timers, которые уже есть в системе

```
systemctl list-timers

systemctl list-timers --all
```

NEXT
следующий запуск

LEFT
сколько осталось

LAST
предыдущий запуск

UNIT
.timer unit

ACTIVATES
что запускает

Это одна из главных команд для повседневной работы с timers.

Шаг 1. Создаём service

/etc/systemd/system/demo-timer.service

```
[Unit]
Description=Demo task for systemd timer

[Service]
Type=oneshot
ExecStart=/usr/bin/date
```

Type=oneshot

Service выполняет одну задачу и после этого завершается.

ExecStart=

Здесь будет реальная команда или путь к вашему скрипту.

```
sudo systemctl daemon-reload
sudo systemctl start demo-
timer.service
journalctl -u demo-timer.service
```

Шаг 2. Создаём timer

/etc/systemd/system/demo-timer.timer

```
[Unit]
Description=Run demo task every day

[Timer]
OnCalendar=*-*-* 02:00:00
Persistent=true

[Install]
WantedBy=timers.target
```

OnCalendar=

Календарное расписание: здесь каждый день в 02:00.

Persistent=true

Если календарный запуск был пропущен во время простоя — timer может догнать его

timers.target

Позволяет включить timer в обычную схему загрузки systemd.

Шаг 3. Включаем и проверяем

```
sudo systemctl daemon-reload

sudo systemctl enable --now demo-
timer.timer

systemctl status demo-timer.timer

systemctl list-timers --all
```

Что происходит?

- daemon-reload — перечитываем unit-файлы
- enable — включаем автозапуск timer
- --now — сразу активируем timer
- list-timers — проверяем следующий запуск

Сначала тестируйте service вручную, потом расписание.

Проверяем OnCalendar до запуска

systemd умеет сам объяснить, как понял расписание

```
systemd-analyze calendar "daily"

systemd-analyze calendar "Mon..Fri 09:00"

systemd-analyze calendar "*-*-* 02:00:00"
```

daily

Каждый день.

Mon..Fri 09:00

По будням в 09:00.

***-*-* 02:00:00**

Каждый день в 02:00.

Очень полезно для проверки сложных календарных выражений.

Timer может считать время не только по календарю

Относительные (monotonic) timers

OnBootSec=5min

Первый запуск через 5 минут после загрузки системы.

OnUnitActiveSec=1h

Следующий запуск через час после предыдущей активации unit.

Можно объединить

Первый запуск после boot, затем повторять с нужным интервалом.

```
[Timer]
OnBootSec=5min
OnUnitActiveSec=1h
```

OnCalendar = календарное время • OnBootSec / OnUnitActiveSec = время относительно состояния системы

Persistent=true и диагностика

Пропущенный запуск

02:00 — timer должен сработать

Но компьютер выключен

08:00 — система включилась

Persistent=true → задача может выполниться после активации timer

```
systemctl status demo-timer.timer

systemctl status demo-timer.service

journalctl -u demo-timer.service

systemctl cat demo-timer.timer
```

Persistent= работает для timers с OnCalendar.

systemd-run: timer без файлов

Полезный бонус и важная команда для LPIC-1

```
sudo systemd-run --on-active=5m
/usr/bin/date
```



Transient units

Systemd временно создаёт .timer и .service для этой команды.

- Удобно для тестов и разовых задач
- Не нужно создавать unit-файлы вручную
- Для постоянной инфраструктурной задачи лучше обычные .service + .timer

```
systemctl list-timers
```

Cron или systemd timer?

cron

✓ Очень короткая конфигурация
✓ Хорош для простых периодических задач
✓ Привычен и широко распространён

Выбирайте, когда расписание простое и дополнительная интеграция systemd не нужна.

systemd timer

✓ systemctl и journalctl
✓ зависимости между units
✓ OnBootSec и интервалы
✓ Persistent=true
✓ единая модель systemd

Выбирайте, когда задача тесно связана с сервисами и администрированием системы.

Не обязательно выбирать «победителя». Linux-администратор должен понимать оба механизма.

Что нужно запомнить

.timer → расписание
.service → действие

Основные параметры

```
OnCalendar=
OnBootSec=
OnUnitActiveSec=
Persistent=true
```

```
systemctl list-timers
systemctl enable --now name.timer
systemctl status name.timer
journalctl -u name.service
systemd-analyze calendar "daily"
systemd-run --on-active=5m /usr/bin/date
```

cron → at → systemd timers

Полезные ссылки

Документация

systemd.timer
https://www.freedesktop.org/software/systemd/man/latest/systemd.timer.html

systemd.time
https://www.freedesktop.org/software/systemd/man/latest/systemd.time.html

systemd-run
https://www.freedesktop.org/software/systemd/man/latest/systemd-run.html

LPIC-1 Objective 107.2
https://wiki.lpi.org/wiki/LPIC-1_Objectives_V5.0

Мои материалы

Сайт
https://solizarevich.github.io/

Telegram
https://t.me/linautonet

ВКонтакте
https://vk.com/linautonet

YouTube
https://youtube.com/@solizarevich

RUTUBE
https://rutube.ru/channel/25269092/

Linux • DevOps • сети • автоматизация